

ASSIGNMENT – 5

Database for Data Science

Name – Sujal Yadav

Section – AD-3

Roll no – 64

Set 1: Easy Level

Q1: Create and switch to the ecommerce_db database

```
> use e_commerce  
< switched to db e_commerce  
e_commerce>|
```

Q2: Create an orders collection by inserting a single order document

```
> // Create an orders collection by inserting a single order document  
> db.Orders.insertOne({  
  order_id: "ORD1001",  
  customer_id: "CUST501",  
  customer_city: "Delhi",  
  order_date: ISODate("2025-02-01"),  
  status: "PLACED",  
  items: [  
    {  
      product_id: "P101",  
      category: "Electronics",  
      price: 25000,  
      quantity: 1  
    }  
  ],  
  payment: {  
    payment_type: "UPI",  
    payment_value: 25000  
  }  
})  
< {  
  acknowledged: true,  
  insertedId: ObjectId('69da9a4e9d9ebd5a00cff538')  
}
```

Q3: Insert multiple order documents into the orders collection

```
> // Insert multiple order documents into the orders collection
> db.Orders.insertMany([
  {
    order_id: "ORD1002",
    customer_id: "CUST502",
    customer_city: "Mumbai",
    order_date: ISODate("2025-02-02"),
    status: "PLACED",
    items: [
      { product_id: "P102", category: "Clothing", price: 2000, quantity: 2 }
    ],
    payment: { payment_type: "CARD", payment_value: 4000 }
  },
  {
    order_id: "ORD1003",
    customer_id: "CUST503",
    customer_city: "Jaipur",
    order_date: ISODate("2025-02-03"),
    status: "PLACED",
    items: [
      { product_id: "P103", category: "Books", price: 500, quantity: 3 }
    ],
    payment: { payment_type: "COD", payment_value: 1500 }
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69da9a959d9ebd5a00cff539'),
```

Q4: Display all order documents stored in the collection

```
> // Display all order documents stored in the collection
> db.orders.find()
< {
  _id: ObjectId('698aae5cebff3d1415e28598'),
  order_id: 'ORD1001',
  customer_id: 'CUST501',
  customer_city: 'Delhi',
  price: 5000
}
{
  _id: ObjectId('698aae69ebff3d1415e28599'),
  order_id: 'ORD1002',
  customer_id: 'CUST502',
  customer_city: 'Noida',
  price: 3000
}
{
  _id: ObjectId('698aae69ebff3d1415e2859a'),
  order_id: 'ORD1003',
  customer_id: 'CUST503',
  customer_city: 'Agra',
  price: 2500
}
{
  _id: ObjectId('698aae69ebff3d1415e2859b'),
  order_id: 'ORD1004',
  customer_id: 'CUST504',
  customer_city: 'Delhi'
}
```

Q5: Display only the following fields for all orders:

- order_id
- customer_city
- status

```

> // Display only the following fields for all orders:
  // order_id
  // customer_city
  // status
> db.Orders.find(
  {},
  { order_id: 1, customer_city: 1, status: 1, _id: 0 }
)
< {
  order_id: 'ORD1001',
  customer_city: 'Delhi',
  status: 'PLACED'
}
{
  order_id: 'ORD1002',
  customer_city: 'Mumbai',
  status: 'PLACED'
}
{
  order_id: 'ORD1003',
  customer_city: 'Jaipur',
  status: 'PLACED'
}

```

Set 2: Moderate Level

Q1: Display all orders placed from a specific customer city

```

> // Display all orders placed from a specific customer city
> db.Orders.find({ customer_city: "Delhi" })
< {
  _id: ObjectId('69da9a4e9d9ebd5a00cff538'),
  order_id: 'ORD1001',
  customer_id: 'CUST501',
  customer_city: 'Delhi',
  order_date: 2025-02-01T00:00:00.000Z,
  status: 'PLACED',
  items: [
    {
      product_id: 'P101',
      category: 'Electronics',
      price: 25000,
      quantity: 1
    }
  ],
  payment: {
    payment_type: 'UPI',
    payment_value: 25000
  }
}

```

Q2: Display orders where the payment value exceeds a given amount

```
> db.Orders.find({
  "payment.payment_value": { $gt: 2000 }
})
< {
  _id: ObjectId('69da9a4e9d9ebd5a00cff538'),
  order_id: 'ORD1001',
  customer_id: 'CUST501',
  customer_city: 'Delhi',
  order_date: 2025-02-01T00:00:00.000Z,
  status: 'PLACED',
  items: [
    {
      product_id: 'P101',
      category: 'Electronics',
      price: 25000,
      quantity: 1
    }
  ],
  payment: {
    payment_type: 'UPI',
    payment_value: 25000
  }
}
{
  _id: ObjectId('69da9a959d9ebd5a00cff539'),
  order_id: 'ORD1002',
  customer_id: 'CUST502',
  customer_city: 'Mumbai',
```

Q3: Update the status of a specific order from PLACED to SHIPPED.

```
> // Update the status of a specific order from PLACED to SHIPPED
> db.Orders.updateOne(
  { order_id: "ORD1001" },
  { $set: { status: "SHIPPED" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Q4: Increase the payment value of all orders made using a specific payment type

```
> // Increase the payment value of all orders made using a specific payment type
> db.Orders.updateMany(
  { "payment.payment_type": "UPI" },
  { $inc: { "payment.payment_value": 500 } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Q5: Add a new field delivery_status with default value "PENDING" to all existing order documents

```
> // Add a new field delivery_status with default value "PENDING"
// to all existing order documents
> db.Orders.updateMany(
  {},
  { $set: { delivery_status: "PENDING" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

Set 3: Hard Level

Q1: Update the status of all orders placed before a specific date


```
> // Update the status of all orders placed before a specific date
> db.Orders.updateMany(
  { order_date: { $lt: ISODate("2025-02-03") } },
  { $set: { status: "DELIVERED" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

Q2: Add a new product item into the items array of a specific order

```
> // Add a new product item into the items array of a specific order
> db.Orders.updateOne(
  { order_id: "ORD1001" },
  {
    $push: {
      items: {
        product_id: "P104",
        category: "Accessories",
        price: 1000,
        quantity: 1
      }
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Q3: Remove a product from the items array based on a product_id

```
> // Remove a product from the items array based on a product_id
> db.Orders.updateOne(
  { order_id: "ORD1001" },
  {
    $pull: {
      items: { product_id: "P101" }
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Q4: Delete one specific order using its order_id

```
> // Delete one specific order using its order_id
> db.Orders.deleteOne({ order_id: "ORD1003" })
< {
  acknowledged: true,
  deletedCount: 1
}
```

Q5: Delete all orders whose payment value is below a given threshold

```
> // Delete all orders whose payment value is below a given threshold
> db.Orders.deleteMany({
  "payment.payment_value": { $lt: 2000 }
})
< {
  acknowledged: true,
  deletedCount: 0
}
```